

NAME:R.KAVIYA

REG NO:192524154

COURSE:CSA0904-PROGRAMMING IN
JAVA

Q1: Student Registration System

ANNEXURE

A university stores student information where:

- Each student must be **unique**.
- A student can enroll in **multiple courses**.

(a) Enumerate three Collection interfaces suitable for this system.

Collection Interface	Justification
Set	Stores unique students without duplicates.
List	Stores multiple courses for each student while maintaining insertion order.
Map	Maps each student to their list of enrolled courses.

(b) Design a Collection Structure (Diagram/Representation)

HashMap<Student, List<String>>

Student List of Courses

Alice -----> [Java, Python]

Bob -----> [DBMS, AI]

Carol -----> [Java, Networks]

(c) Java Program using HashSet and HashMap with Generics

```

Main.java
1 import java.util.*;
2 class Student {
3     int id;
4     String name;
5
6     Student(int id, String name) {
7         this.id = id;
8         this.name = name;
9     }
10
11     public int hashCode() {
12         return id;
13     }
14
15     public boolean equals(Object obj) {
16         Student s = (Student) obj;
17         return this.id == s.id;
18     }
19
20     public String toString() {
21         return id + " - " + name;
22     }
23 }
24 public class Main {
25     public static void main(String[] args) {
26         HashSet<Student> students = new HashSet<>();
27         Student s1 = new Student(101, "Alice");
28         Student s2 = new Student(102, "Bob");
29         Student s3 = new Student(103, "Carol");
30         students.add(s1);
31         students.add(s2);
32         students.add(s3);
33         HashMap<Student, List<String>> courseMap = new HashMap<>();
34
35         courseMap.put(s1, Arrays.asList("Java", "Python"));
36         courseMap.put(s2, Arrays.asList("DBMS", "AI"));
37         courseMap.put(s3, Arrays.asList("Java", "Networks"));
38
39         Iterator<Student> it = students.iterator();
40
41         while (it.hasNext()) {
42             Student s = it.next();
43             System.out.println("Student : " + s);
44             System.out.println("Courses : " + courseMap.get(s));
45             System.out.println();
46         }
47     }
48 }

```

```

Student : 101 - Alice
Courses : [Java, Python]

Student : 102 - Bob
Courses : [DBMS, AI]

Student : 103 - Carol
Courses : [Java, Networks]

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Q2: E-Commerce Cart System

ANNEXURE

An online shopping system maintains a cart where:

- Products can be added.
- Products can be removed.
- Products are displayed in the order they were added.

(a) List three Collection classes applicable for cart management.

Collection Class	Justification
ArrayList	Stores cart items in insertion order and allows easy access.
LinkedList	Efficient for frequent insertion and deletion of cart items.
Vector	Similar to ArrayList but synchronized (thread-safe).

(b) Represent how cart items are stored using a List-based structure.

ArrayList (Cart)

Index	Item
0	Laptop
1	Mouse
2	Keyboard
3	Headphones

(c) Java Program using ArrayList and Iterator

```
Main.java
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         ArrayList<String> cart = new ArrayList<>();
7
8
9         cart.add("Laptop");
10        cart.add("Mouse");
11        cart.add("Keyboard");
12        cart.add("Headphones");
13
14
15        cart.remove("Mouse");
16
17
18
19        System.out.println("Items in Cart:");
20
21        Iterator<String> it = cart.iterator();
22
23        while (it.hasNext()) {
24            System.out.println(it.next());
25        }
26    }
27 }
```

Items in Cart:
Laptop
Keyboard
Headphones

...Program finished with exit code 0
Press ENTER to exit console.

Q3: Library Management System

ANNEXURE

A library stores books where:

- Each book has a unique ISBN number.
- Books should be retrieved quickly.
- The system should display all stored books.

(a) Name three Map implementations in Java.

Map Implementation	Justification
HashMap	Fast insertion and retrieval using key-value pairs.
LinkedHashMap	Maintains insertion order while storing key-value pairs.
TreeMap	Stores keys in sorted order.

(b) Design a schema-like representation using Map for storing ISBN and book details.

```
HashMap<String, String>
ISBN (Key)   Book Details (Value)
101    --> Java Programming
102    --> Data Structures
103    --> Database Systems
```

(c) Java Program using HashMap

```
1- import java.util.*;
2
3- public class Main{
4-     public static void main(String[] args) {
5
6         HashMap<String, String> books = new HashMap<>();
7
8         books.put("101", "Java Programming");
9         books.put("102", "Data Structures");
10        books.put("103", "Database Systems");
11
12        System.out.println("Book with ISBN 102: " + books.get("102"));
13
14        System.out.println("\nAll Books:");
15
16        for (Map.Entry<String, String> entry : books.entrySet()) {
17            System.out.println("ISBN: " + entry.getKey() +
18                               " Book: " + entry.getValue());
19        }
20    }
21 }
22
23
24 }
```

```
All Books:
ISBN: 101 Book: Java Programming
ISBN: 102 Book: Data Structures
ISBN: 103 Book: Database Systems

...Program finished with exit code 0
Press ENTER to exit console.
```

Q4: Employee Data Processing

ANNEXURE

A company processes employee records and filters employees based on salary.

(a) List three advantages of Generics in Collections.

Advantage	Justification
Type Safety	Prevents storing the wrong type of objects in a collection.
No Type Casting	Objects can be retrieved directly without explicit casting.
Code Reusability	The same collection code can work with different object types.

(b) Represent how employee data is stored using List with Generics.

ArrayList<Employee>

Index	Employee
0	(101, Alice, 60000)
1	(102, Bob, 45000)
2	(103, Carol, 70000)

(c) Java Program using Iterator

```
1- import java.util.*;
2- class Employee {
3-     int id;
4-     String name;
5-     double salary;
6-     Employee(int id, String name, double salary) {
7-         this.id = id;
8-         this.name = name;
9-         this.salary = salary;
10-    }
11-    public String toString() {
12-        return id + " - " + name + " - " + salary;
13-    }
14- }
15- public class Main {
16-     public static void main(String[] args) {
17-         ArrayList<Employee> empList = new ArrayList<>();
18-         empList.add(new Employee(101, "Alice", 60000));
19-         empList.add(new Employee(102, "Bob", 45000));
20-         empList.add(new Employee(103, "Carol", 70000));
21-         System.out.println("Employees with Salary > 50000:");
22-         Iterator<Employee> it = empList.iterator();
23-         while (it.hasNext()) {
24-             Employee e = it.next();
25-             if (e.salary > 50000) {
26-                 System.out.println(e);
27-             }
28-         }
29-     }
30- }
```

```
Employees with Salary > 50000:
101 - Alice - 60000.0
103 - Carol - 70000.0
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Q5: Online Voting System

ANNEXURE

An online voting system:

- Allows only one vote per user.
- Prevents duplicate voting.
- Counts votes efficiently.

(a) Identify three suitable Collection types for this system.

Collection Type	Justification
HashSet	Stores unique voter IDs and prevents duplicate voting.
HashMap	Stores candidates and their vote counts.
ArrayList	Stores the list of candidates.

(b) Design a structure using Set and Map for voters and vote counting.

HashSet<String>

Voter IDs

V101

V102

V103

HashMap<String, Integer>

Candidate	Votes
-----------	-------

Alice	2
-------	---

Bob	1
-----	---

(c) Java Program using HashSet and HashMap

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         HashSet<String> voters = new HashSet<>();
5
6         HashMap<String, Integer> votes = new HashMap<>();
7
8         votes.put("Alice", 0);
9         votes.put("Bob", 0);
10
11         castVote(voters, votes, "V101", "Alice");
12         castVote(voters, votes, "V102", "Bob");
13         castVote(voters, votes, "V103", "Alice");
14
15
16         castVote(voters, votes, "V101", "Bob");
17
18         System.out.println("\nVoting Results:");
19         for (Map.Entry<String, Integer> entry : votes.entrySet()) {
20             System.out.println(entry.getKey() + " : " + entry.getValue());
21         }
22     }
23
24     public static void castVote(HashSet<String> voters,
25                                HashMap<String, Integer> votes,
26                                String voterId,
27                                String candidate) {
28
29         if (voters.add(voterId)) {
30             votes.put(candidate, votes.get(candidate) + 1);
31             System.out.println(voterId + " voted for " + candidate);
32         } else {
33             System.out.println(voterId + " has already voted!");
34         }
35     }
36 }
37 }
```

```
V101 voted for Alice
V102 voted for Bob
V103 voted for Alice
V101 has already voted!

Voting Results:
Bob : 1
Alice : 2

...Program finished with exit code 0
Press ENTER to exit console.
```